

Web Scrapping data (Data Collection)

CODING:

```
# Import Dependencies
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.common.keys import Keys
```

```
from selenium.webdriver.support.ui import WebDriverWait
```

```
from selenium.webdriver.support import expected_conditions as EC
```

```
from selenium.common.exceptions import NoSuchElementException, TimeoutException,  
WebDriverException
```

```
from time import sleep
```

```
import pandas as pd
```

```
# Setup WebDriver with ChromeOptions
```

```
chrome_options = webdriver.ChromeOptions()
```

```
chrome_options.add_experimental_option("detach", True)
```

```
driver = webdriver.Chrome(options=chrome_options)
```

```
driver.get("https://twitter.com/login")
```

```
# Twitter credentials (Insert your own)
```

```
username_str = ""
```

```
password_str = ""
```

```
# Setup the login
```

```
try:
```

```
    username = WebDriverWait(driver, 10).until(
```

```
        EC.presence_of_element_located((By.XPATH, "//input[@name='text']"))
```

```
    )
```

```

username.send_keys(username_str)

next_button = driver.find_element(By.XPATH, "//span[contains(text(),'Next')]")
next_button.click()

except (NoSuchElementException, TimeoutException, WebDriverException) as e:
    print(f"Error during login: {e}")
    driver.quit()

try:
    password = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.XPATH, "//input[@name='password']"))
    )
    password.send_keys(password_str)
    log_in = driver.find_element(By.XPATH, "//span[contains(text(),'Log in')]")
    log_in.click()
except (NoSuchElementException, TimeoutException, WebDriverException) as e:
    print(f"Error during login: {e}")
    driver.quit()

# Search for the hashtag #BlackLivesMatter

try:
    search_box = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.XPATH, "//input[@data-
        testid='SearchBox_Search_Input']"))
    )
    search_box.send_keys("Black Lives Matter")
    search_box.send_keys(Keys.ENTER)
except (NoSuchElementException, TimeoutException, WebDriverException) as e:
    print(f"Error during search: {e}")
    driver.quit()

# Click on the "People" tab

try:
    people_tab = WebDriverWait(driver, 10).until(

```

```

        EC.element_to_be_clickable((By.XPATH, "//span[contains(text(),'People')]"))
    )
    people_tab.click()
except (NoSuchElementException, TimeoutException, WebDriverException) as e:
    print(f"Error during switching to people: {e}")
    driver.quit()

# Click on the first profile in the "People" tab
try:
    first_profile = WebDriverWait(driver, 10).until(
        EC.element_to_be_clickable((By.XPATH, "//*[@id='react-
root']/div/div/div[2]/main/div/div/div/div/div/div[3]/section/div/div/div[1]/div/div/button/div/div[2]/div[1]/div[1]/div/div[1]/a/div/div[1]/span/span[1]"))
    )
    first_profile.click()
except (NoSuchElementException, TimeoutException, WebDriverException) as e:
    print(f"Error during profile selection: {e}")
    driver.quit()

# Initialize lists to store tweet data
UserTags = []
TimeStamps = []
Tweets = []
Replies = []
Retweets = []
Likes = []

# Fetch tweets from the profile
try:
    sleep(10)
    last_height = driver.execute_script("return document.body.scrollHeight")
    while True:
        articles = driver.find_elements(By.XPATH, "//article[@data-testid='tweet']")

```

for article in articles:

try:

UserTag = article.find_element(By.XPATH, "//*[@data-testid='User-Name']").text

TimeStamp = article.find_element(By.XPATH, "//*[@time]").get_attribute('datetime')

Tweet = article.find_element(By.XPATH, "//*[@data-testid='tweetText']").text

Reply = article.find_element(By.XPATH, "//*[@data-testid='reply']").text

Retweet = article.find_element(By.XPATH, "//*[@data-testid='retweet']").text

Like = article.find_element(By.XPATH, "//*[@data-testid='like']").text

UserTags.append(UserTag)

TimeStamps.append(TimeStamp)

Tweets.append(Tweet)

Replies.append(Reply)

Retweets.append(Retweet)

Likes.append(Like)

except NoSuchElementException as e:

print(f"Error: {e}")

driver.execute_script('window.scrollTo(0,document.body.scrollHeight);')

sleep(5)

new_height = driver.execute_script("return document.body.scrollHeight")

if new_height == last_height: # Break if no new tweets are loaded

break

last_height = new_height

Optionally break after a certain number of tweets

if len(Tweets) > 4000: # Adjust as needed

break

Print the number of collected tweets

print(len(UserTags), len(TimeStamps), len(Tweets), len(Replies), len(Retweets), len(Likes))

```

# Save data to a DataFrame and CSV file
df = pd.DataFrame(zip(UserTags, TimeStamps, Tweets, Replies, Retweets, Likes),
                  columns=['UserTags', 'TimeStamps', 'Tweets', 'Replies', 'Retweets', 'Likes'])

df.head()

# Save the DataFrame to a CSV file in the current working directory
df.to_csv("tweets_live.csv", index=False)

print("Tweets have been saved to 'tweets_live.csv'")

except WebDriverException as e:
    print(f"Error during tweet fetching: {e}")
finally:
    # Close the driver
    driver.quit()

```

Sentiment analysis using (Twitter)

```

import pandas as pd
import squarify
import collections
import re
from textblob import TextBlob
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from collections import Counter

```

```
import seaborn as sns
```

```
df = pd.read_csv("tweets_live9.csv")
```

```
df.head()
```

```
df['TimeStamps'] = pd.to_datetime(df['TimeStamps'])
```

```
def clean_tweet(tweet):
```

```
    if isinstance(tweet, str):
```

```
        tweet = re.sub(r'http\S+', "", tweet) # Remove URLs
```

```
        tweet = re.sub(r'@\S+', "", tweet)    # Remove mentions
```

```
        tweet = re.sub(r'#\S+', "", tweet)    # Remove hashtags
```

```
        tweet = re.sub(r'^A-Za-z0-9\s|', "", tweet) # Remove special characters
```

```
        tweet = tweet.lower() # Convert to lowercase
```

```
    else:
```

```
        tweet = "" # If the tweet is not a string, convert to an empty string
```

```
    return tweet
```

```
df['Cleaned_Tweets'] = df['Tweets'].apply(clean_tweet)
```

```
# Process the numeric columns
```

```
def process_k_values(value):
```

```
    value = str(value)
```

```
    if 'K' in value:
```

```
        return float(value.replace('K', "")) * 1000
```

```
    return float(value)
```

```
df['Replies'] = df['Replies'].apply(process_k_values)
```

```
df['Retweets'] = df['Retweets'].apply(process_k_values)
df['Likes'] = df['Likes'].apply(process_k_values)
```

```
df
```

```
def get_sentiment(tweet):
    analysis = TextBlob(tweet)
    return analysis.sentiment.polarity
```

```
df['Sentiment'] = df['Cleaned_Tweets'].apply(get_sentiment)
```

```
df
```

```
df.describe()
```

```
# Combine all cleaned tweets into one large string
all_words = ' '.join([tweet for tweet in df['Cleaned_Tweets']])
```

```
# Generate the word cloud
wordcloud = WordCloud(width=800, height=400, random_state=21,
max_font_size=110).generate(all_words)
```

```
# Plot the word cloud
plt.figure(figsize=(10, 7))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis('off')
plt.title('Word Cloud of #BlackLivesMatter Tweets')
plt.show()
```

```

# Create sentiment labels

df['Sentiment_Label'] = df['Sentiment'].apply(lambda x: 'Positive' if x > 0 else ('Negative' if x < 0 else
'Neutral'))

# Prepare data for the treemap

sentiment_counts = df['Sentiment_Label'].value_counts().reset_index()

sentiment_counts.columns = ['Sentiment', 'Count']

# Calculate percentages

total_count = sentiment_counts['Count'].sum()

sentiment_counts['Percentage'] = (sentiment_counts['Count'] / total_count) * 100

sentiment_counts['Label'] = sentiment_counts.apply(lambda x: f"{x['Sentiment']}\n{x['Count']}"
({x['Percentage']:.2f}%)", axis=1)


# Plot the treemap with percentages

plt.figure(figsize=(12, 8))

colors = ['green', 'red', 'blue']

squarify.plot(sizes=sentiment_counts['Count'], label=sentiment_counts['Label'], color=colors,
alpha=.8)

plt.title('Sentiment Distribution of #BlackLivesMatter Tweets')

plt.axis('off')

plt.show()


# Function to extract most common words

def get_most_common_words(text_series, num_words=464):

    all_words = ' '.join(text_series).split()

    common_words = Counter(all_words).most_common(num_words)

    return dict(common_words)


# Extract words for each sentiment

positive_words = get_most_common_words(df[df['Sentiment_Label'] ==
'Positive']['Cleaned_Tweets'])

neutral_words = get_most_common_words(df[df['Sentiment_Label'] == 'Neutral']['Cleaned_Tweets'])

```

```
negative_words = get_most_common_words(df[df['Sentiment_Label'] ==  
'Negative']['Cleaned_Tweets'])
```

```
# Plot word clouds for each sentiment
```

```
plt.figure(figsize=(18, 6))
```

```
plt.subplot(1, 3, 1)
```

```
plt.imshow(WordCloud(width=600, height=400,  
background_color='white').generate_from_frequencies(positive_words))
```

```
plt.title('Positive Words')
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 2)
```

```
plt.imshow(WordCloud(width=600, height=400,  
background_color='white').generate_from_frequencies(neutral_words))
```

```
plt.title('Neutral Words')
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 3)
```

```
plt.imshow(WordCloud(width=600, height=400,  
background_color='white').generate_from_frequencies(negative_words))
```

```
plt.title('Negative Words')
```

```
plt.axis('off')
```

```
plt.show()
```

```
# Ensure all tweets are strings and handle non-string entries
```

```
df['Tweets'] = df['Tweets'].astype(str)
```

```
# Extract hashtags from the original tweets
```

```
df['Hashtags'] = df['Tweets'].apply(lambda x: re.findall(r'#\w+', x))
```

```
# Flatten the list of hashtags and count frequencies
```

```

all_hashtags = [hashtag for sublist in df['Hashtags'] for hashtag in sublist]
hashtag_counts = collections.Counter(all_hashtags)

# Convert to DataFrame for easier handling
hashtag_df = pd.DataFrame(hashtag_counts.most_common(10), columns=['Hashtag', 'Count'])

# Plot the most common hashtags
plt.figure(figsize=(10, 6))
plt.barh(hashtag_df['Hashtag'], hashtag_df['Count'], color='skyblue')
plt.xlabel('Count')
plt.title('Top 10 Hashtags in Black Lives Matter Movement Tweets')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()

# Extract mentions from the original tweets
df['Mentions'] = df['Tweets'].apply(lambda x: re.findall(r'@\w+', x))

# Flatten the list of mentions and count frequencies
all_mentions = [mention for sublist in df['Mentions'] for mention in sublist]
mention_counts = collections.Counter(all_mentions)

# Convert to DataFrame for easier handling
mention_df = pd.DataFrame(mention_counts.most_common(10), columns=['Mention', 'Count'])

# Plot the most common mentions as a pie chart
plt.figure(figsize=(10, 6))
plt.pie(mention_df['Count'], labels=mention_df['Mention'], autopct='%1.1f%%',
        colors=plt.cm.Paired(range(len(mention_df))))
plt.title('Top 10 Mentions in #BlackLivesMatter Tweets')
plt.axis('equal') # Equal aspect ratio ensures the pie is drawn as a circle
plt.show()

```

```
# Calculate the length of words in each tweet
```

```
df['Word_Lengths'] = df['Cleaned_Tweets'].apply(lambda x: len(x.split()))
```

```
# Plot the histogram
```

```
plt.figure(figsize=(10, 6))
```

```
plt.hist(df['Word_Lengths'], bins=30, color='skyblue', edgecolor='black')
```

```
plt.title('Histogram of Word Lengths in Tweets')
```

```
plt.xlabel('Word Length')
```

```
plt.ylabel('Frequency')
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Create a box plot for word lengths based on sentiment
```

```
plt.figure(figsize=(10, 6))
```

```
ax = sns.boxplot(x='Sentiment_Label', y='Word_Lengths', data=df, palette="rocket")
```

```
ax.set_title('Box Plot of Word Lengths by Sentiment')
```

```
ax.set_xlabel('Sentiment')
```

```
ax.set_ylabel('Word Lengths')
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Extract the date from the TimeStamp
```

```
df['Date'] = df['TimeStamps'].dt.date
```

```
# Plot the trend of tweets posted each day
```

```
daily_tweets = df.groupby('Date').size()
```

```
plt.figure(figsize=(12, 6))
```

```
daily_tweets.plot(kind='line', color='skyblue')
```

```
plt.title('Trend of Tweets Posted Each Day From The Organisation')
```

```
plt.xlabel('Date')
plt.ylabel('Number of Tweets')
plt.grid(True)
plt.show()
```

```
# Plot the distribution of likes, retweets, and replies
```

```
plt.figure(figsize=(12, 6))
```

```
# Likes
```

```
plt.subplot(1, 3, 1)
plt.hist(df['Likes'], bins=30, color='lightcoral', edgecolor='black')
plt.title('Distribution of Likes')
plt.xlabel('Likes')
plt.ylabel('Frequency')
plt.grid(True)
```

```
# Retweets
```

```
plt.subplot(1, 3, 2)
plt.hist(df['Retweets'], bins=30, color='lightgreen', edgecolor='black')
plt.title('Distribution of Retweets')
plt.xlabel('Retweets')
plt.ylabel('Frequency')
plt.grid(True)
```

```
# Replies
```

```
plt.subplot(1, 3, 3)
plt.hist(df['Replies'], bins=30, color='lightblue', edgecolor='black')
plt.title('Distribution of Replies')
plt.xlabel('Replies')
plt.ylabel('Frequency')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Calculate the correlation matrix
```

```
corr_matrix = df[['Likes', 'Retweets', 'Replies']].corr()
```

```
# Plot the heatmap with enhancements
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(corr_matrix, annot=True, cmap='YlGnBu', fmt='.2f', linewidths=.5)
```

```
plt.title('Correlation Matrix of Likes, Retweets, and Replies', fontsize=18, color='navy')
```

```
plt.xticks(rotation=45)
```

```
plt.yticks(rotation=0)
```

```
plt.xlabel('Metrics', fontsize=14)
```

```
plt.ylabel('Metrics', fontsize=14)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Extract the hour from the TimeStamps
```

```
df['Hour'] = df['TimeStamps'].dt.hour
```

```
# Calculate the number of tweets per hour
```

```
tweets_per_hour = df.groupby('Hour').size()
```

```
# Create a single figure with subplots
```

```
plt.figure(figsize=(18, 12))
```

```
# Line plot for number of Tweets per Hour
```

```
plt.subplot(2, 2, 1)
plt.plot(tweets_per_hour, marker='o', linestyle='-', color='b')
plt.title("Number of Tweets by Hour", fontsize=19, color='k')
plt.xlabel('Hour of Day')
plt.ylabel('Number of Tweets')
```

```
# Box plot for Likes by Hour
plt.subplot(2, 2, 2)
sns.boxplot(x='Hour', y='Likes', data=df, palette='cool')
plt.title("Likes by Hour", fontsize=19, color='k')
plt.xlabel('Hour of Day')
plt.ylabel('Likes')
```

```
# Box plot for Retweets by Hour
plt.subplot(2, 2, 3)
sns.boxplot(x='Hour', y='Retweets', data=df, palette='cool')
plt.title("Retweets by Hour", fontsize=19, color='k')
plt.xlabel('Hour of Day')
plt.ylabel('Retweets')
```

```
# Box plot for Replies by Hour
plt.subplot(2, 2, 4)
sns.boxplot(x='Hour', y='Replies', data=df, palette='cool')
plt.title("Replies by Hour", fontsize=19, color='k')
plt.xlabel('Hour of Day')
plt.ylabel('Replies')
```

```
plt.tight_layout()
plt.show()
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, roc_curve, auc
import xgboost as xgb
from sklearn.preprocessing import LabelEncoder
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import Trainer, TrainingArguments
import torch
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
import numpy as np
from sklearn.preprocessing import label_binarize
from itertools import cycle

# Split the data
X = df['Cleaned_Tweets']
y = df['Sentiment_Label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Vectorize the text data
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# Train the Logistic Regression model
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_tfidf, y_train)

# Evaluate the model
y_pred_log_reg = log_reg.predict(X_test_tfidf)
```

```

print("Logistic Regression:")
print(classification_report(y_test, y_pred_log_reg))
print("Accuracy:", accuracy_score(y_test, y_pred_log_reg))

# Encode sentiment labels to numeric values
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

# Train the XGBoost model
xgb_model = xgb.XGBClassifier(eval_metric='mlogloss')
xgb_model.fit(X_train_tfidf, y_train_encoded)

# Evaluate the model
y_pred_xgb = xgb_model.predict(X_test_tfidf)
print("XGBoost:")
print(classification_report(y_test_encoded, y_pred_xgb, target_names=label_encoder.classes_))
print("Accuracy:", accuracy_score(y_test_encoded, y_pred_xgb))

# Encode the labels
label_encoder = LabelEncoder()
df['Sentiment_Label'] = label_encoder.fit_transform(df['Sentiment_Label'])
y = df['Sentiment_Label']

# Tokenize the data
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def encode_data(texts, labels, max_length=128):
    input_ids = []
    attention_masks = []

```

```

for text in texts:
    encoded = tokenizer.encode_plus(
        text,
        add_special_tokens=True,
        max_length=max_length,
        truncation=True,
        padding='max_length',
        return_attention_mask=True,
        return_tensors='pt'
    )
    input_ids.append(encoded['input_ids'])
    attention_masks.append(encoded['attention_mask'])

return torch.cat(input_ids, dim=0), torch.cat(attention_masks, dim=0), torch.tensor(labels,
dtype=torch.long)

# Convert texts to list and labels to NumPy array
X_train_list = X_train.tolist()
X_test_list = X_test.tolist()
y_train_encoded = label_encoder.transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

X_train_ids, X_train_masks, y_train_tensor = encode_data(X_train_list, y_train_encoded)
X_test_ids, X_test_masks, y_test_tensor = encode_data(X_test_list, y_test_encoded)

# Create a custom dataset
class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, input_ids, attention_masks, labels):
        self.input_ids = input_ids
        self.attention_masks = attention_masks
        self.labels = labels

```

```

def __len__(self):
    return len(self.labels)

def __getitem__(self, idx):
    return {
        'input_ids': self.input_ids[idx],
        'attention_mask': self.attention_masks[idx],
        'labels': self.labels[idx]
    }

train_dataset = CustomDataset(X_train_ids, X_train_masks, y_train_tensor)
test_dataset = CustomDataset(X_test_ids, X_test_masks, y_test_tensor)

# Define the BERT model
model = BertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=len(label_encoder.classes_))

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    evaluation_strategy='epoch'
)

# Train the model
trainer = Trainer(
    model=model,

```

```

    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset
)

trainer.train()

# Evaluate the model
predictions = trainer.predict(test_dataset)
preds = predictions.predictions.argmax(-1)
print("BERT:")
print(classification_report(y_test_encoded, preds, target_names=label_encoder.classes_))
print("Accuracy:", accuracy_score(y_test_encoded, preds))

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Vectorize the text data
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# Train the Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_tfidf, y_train)

# Evaluate the Random Forest model
y_pred_rf = rf_model.predict(X_test_tfidf)
print("Random Forest:")
print(classification_report(y_test, y_pred_rf))
print("Accuracy:", accuracy_score(y_test, y_pred_rf))

```

```
# Train the Decision Tree model

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_tfidf, y_train)

# Evaluate the Decision Tree model

y_pred_dt = dt_model.predict(X_test_tfidf)
print("Decision Tree:")
print(classification_report(y_test, y_pred_dt))
print("Accuracy:", accuracy_score(y_test, y_pred_dt))

# Creating DataFrames for classification reports

rf_classification_report = classification_report(y_test, y_pred_rf, output_dict=True)
dt_classification_report = classification_report(y_test, y_pred_dt, output_dict=True)

rf_df = pd.DataFrame(rf_classification_report).transpose()
dt_df = pd.DataFrame(dt_classification_report).transpose()

# Convert labels to string type to avoid mix of types

y_test_str = y_test.astype(str)
y_pred_log_reg_str = y_pred_log_reg.astype(str)
y_pred_rf_str = y_pred_rf.astype(str)
y_pred_dt_str = y_pred_dt.astype(str)

# For Logistic Regression

log_reg_accuracy = accuracy_score(y_test_str, y_pred_log_reg_str)
log_reg_classification_report = classification_report(y_test_str, y_pred_log_reg_str,
output_dict=True)

# For XGBoost

xgb_accuracy = accuracy_score(y_test_encoded, y_pred_xgb)
```

```
xgb_classification_report = classification_report(y_test_encoded, y_pred_xgb, output_dict=True,
target_names=label_encoder.classes_)
```

```
# For Random Forest
```

```
rf_accuracy = accuracy_score(y_test_str, y_pred_rf_str)
```

```
rf_classification_report = classification_report(y_test_str, y_pred_rf_str, output_dict=True)
```

```
# For Decision Tree
```

```
dt_accuracy = accuracy_score(y_test_str, y_pred_dt_str)
```

```
dt_classification_report = classification_report(y_test_str, y_pred_dt_str, output_dict=True)
```

```
# Manually input the logs here. Example format:
```

```
logs = [  
    {'loss': 1.199700, 'epoch': 1.0},  
    {'eval_loss': 1.109024, 'epoch': 1.0},  
    {'loss': 1.063300, 'epoch': 2.0},  
    {'eval_loss': 0.973764, 'epoch': 2.0},  
    {'loss': 0.893600, 'epoch': 3.0},  
    {'eval_loss': 0.928286, 'epoch': 3.0}  
]
```

```
def plot_metrics(logs, model_name):
```

```
    # Separate training and evaluation logs
```

```
    train_logs = [log for log in logs if 'loss' in log and 'eval_loss' not in log]
```

```
    eval_logs = [log for log in logs if 'eval_loss' in log]
```

```
    # Extract relevant metrics if available
```

```
    train_epochs = [log['epoch'] for log in train_logs]
```

```
    train_loss = [log['loss'] for log in train_logs]
```

```
    eval_epochs = [log['epoch'] for log in eval_logs]
```

```
    eval_loss = [log['eval_loss'] for log in eval_logs]
```

```

plt.figure(figsize=(10, 5))
plt.title(f'Training and Validation Loss for Model {model_name}', fontsize=16)

# Plotting Loss
if train_epochs and train_loss:
    plt.plot(train_epochs, train_loss, label='Training Loss', marker='o', color='blue')
if eval_epochs and eval_loss:
    plt.plot(eval_epochs, eval_loss, label='Validation Loss', marker='o', color='red')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()

# Plot the metrics
plot_metrics(logs, "BERT")

# Creating DataFrames for classification reports
log_reg_df = pd.DataFrame(log_reg_classification_report).transpose()
xgb_df = pd.DataFrame(xgb_classification_report).transpose()
rf_df = pd.DataFrame(rf_classification_report).transpose()
dt_df = pd.DataFrame(dt_classification_report).transpose()

# Plotting the classification reports as subplots
fig, axes = plt.subplots(2, 2, figsize=(20, 20))

def plot_classification_report(ax, df, model_name):
    sns.heatmap(df.iloc[:-1, :].T, annot=True, cmap='Blues', fmt='.2f', ax=ax)
    ax.set_title(f'{model_name} - Classification Report', fontsize=16)
    ax.set_xlabel('Metrics', fontsize=14)
    ax.set_ylabel('Classes', fontsize=14)

```

```

# Plot Logistic Regression classification report
plot_classification_report(axes[0, 0], log_reg_df, 'Logistic Regression')

# Plot XGBoost classification report
plot_classification_report(axes[0, 1], xgb_df, 'XGBoost')

# Plot Random Forest classification report
plot_classification_report(axes[1, 0], rf_df, 'Random Forest')

# Plot Decision Tree classification report
plot_classification_report(axes[1, 1], dt_df, 'Decision Tree')

plt.tight_layout()
plt.show()

# Sample input data
data = {
    'TEXT INPUT': [
        "The smartphone's battery life is exceptional and lasts all day.",
        "My new laptop is slow and crashes frequently.",
        "These headphones have average sound quality but are very comfortable.",
        "The smartwatch tracks my fitness activities accurately.",
        "The vacuum cleaner is efficient and works on all surfaces well."
    ],
    'EXPECT RESULT': ["Positive", "Negative", "Neutral", "Positive", "Positive"]
}

# Create DataFrame
df = pd.DataFrame(data)

# Initialize BERT tokenizer and model

```

```

bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=3)

# Function to get BERT model outputs
def get_bert_output(texts, tokenizer, model):
    inputs = tokenizer(texts, return_tensors='pt', padding=True, truncation=True, max_length=128)
    outputs = model(**inputs)
    probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1).detach().numpy()
    predicted_labels = np.argmax(probabilities, axis=1)
    return predicted_labels, probabilities

# Get BERT outputs
bert_preds, bert_probs = get_bert_output(df['TEXT INPUT'].tolist(), bert_tokenizer, bert_model)
df['BERT OUT'] = label_encoder.inverse_transform(bert_preds)
df['BERT PROB'] = bert_probs.max(axis=1)

# XGBoost predictions and probabilities
xgb_probs = xgb_model.predict_proba(tfidf.transform(df['TEXT INPUT']))
xgb_preds = np.argmax(xgb_probs, axis=1)
df['XGBOOST OUT'] = label_encoder.inverse_transform(xgb_preds)
df['XGBOOST PROB'] = xgb_probs.max(axis=1)

# Logistic Regression predictions and probabilities
log_reg_probs = log_reg.predict_proba(tfidf.transform(df['TEXT INPUT']))
log_reg_preds = np.argmax(log_reg_probs, axis=1)
df['LOGISTIC OUT'] = label_encoder.inverse_transform(log_reg_preds)
df['LOGISTIC PROB'] = log_reg_probs.max(axis=1)

# Display the DataFrame
print(df)

```

```

# Assuming y_test and model probability predictions are already defined

# y_test is the true labels, and models is a dictionary with model names and their probability
# predictions

# Binarize the output
y_test_binarized = label_binarize(y_test, classes=np.unique(y_test))
n_classes = y_test_binarized.shape[1]

# Model probability predictions (ensure these are already computed)
# y_score_log_reg, y_score_xgb, y_score_rf, y_score_dt, y_score_bert
# Example: y_score_log_reg = log_reg_ovr.predict_proba(X_test_tfidf)

# Logistic Regression
y_score_log_reg = log_reg.decision_function(X_test_tfidf)

# XGBoost
y_score_xgb = xgb_model.predict_proba(X_test_tfidf)

# BERT
y_score_bert = torch.nn.functional.softmax(torch.tensor(predictions.predictions), dim=-1).numpy()

# Random Forest
y_score_rf = rf_model.predict_proba(X_test_tfidf)

# Decision Tree
y_score_dt = dt_model.predict_proba(X_test_tfidf)

models = {
    'Logistic Regression': y_score_log_reg,
    'XGBoost': y_score_xgb,
    'Random Forest': y_score_rf,
    'Decision Tree': y_score_dt,
    'BERT': y_score_bert
}

```

```

# Compute ROC curve and ROC area for each class and each model

fpr = dict()
tpr = dict()
roc_auc = dict()

for model_name, y_score in models.items():
    if len(y_score.shape) == 1:
        y_score = label_binarize(y_score, classes=np.arange(n_classes))

    for i in range(n_classes):
        fpr[model_name, i], tpr[model_name, i], _ = roc_curve(y_test_binarized[:, i], y_score[:, i])
        roc_auc[model_name, i] = auc(fpr[model_name, i], tpr[model_name, i])

# Compute micro-average ROC curve and ROC area
fpr[model_name, "micro"], tpr[model_name, "micro"], _ = roc_curve(y_test_binarized.ravel(),
y_score.ravel())
roc_auc[model_name, "micro"] = auc(fpr[model_name, "micro"], tpr[model_name, "micro"])

# Plot ROC curves for the models
plt.figure(figsize=(12, 8))
colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'darkred', 'purple'])
sns.set(style="whitegrid")

for (model_name, color) in zip(models.keys(), colors):
    plt.plot(fpr[model_name, "micro"], tpr[model_name, "micro"], color=color, lw=2,
             label=f'{model_name} (area = {roc_auc[model_name, "micro"]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=14)
plt.ylabel('True Positive Rate', fontsize=14)
plt.title('Receiver Operating Characteristic (ROC) Curve', fontsize=16)

```

```
plt.legend(loc="lower right", fontsize=12)
plt.show()
```

```
# Compute accuracy for each model
```

```
accuracies = {}
```

```
for model_name, y_score in models.items():
```

```
    if len(y_score.shape) > 1:
```

```
        y_pred = np.argmax(y_score, axis=1)
```

```
    else:
```

```
        y_pred = y_score
```

```
    accuracies[model_name] = accuracy_score(y_test, y_pred)
```

```
# Plot accuracy comparison
```

```
plt.figure(figsize=(10, 6))
```

```
sns.barplot(x=list(accuracies.keys()), y=list(accuracies.values()), palette='viridis')
```

```
plt.xlabel('Model', fontsize=14)
```

```
plt.ylabel('Accuracy', fontsize=14)
```

```
plt.title('Comparison of Model Accuracies', fontsize=16)
```

```
plt.ylim(0, 1)
```

```
plt.show()
```